

Enhancing Graph Representation Learning with WalkLM for Effective Community Detection

Hyun Lee*[†]
Trinity College
Hartford, Connecticut, USA
hyun.lee@trincoll.edu

Chris Yi*
Trinity College
Hartford, Connecticut, USA
christopher.yi@trincoll.edu

Maminur Islam*
Trinity College
Hartford, Connecticut, USA
maminur.islam@trincoll.edu

Chandranil (Nil) Chakrabortii
Trinity College
Hartford, Connecticut, USA
nil.chakrabortii@trincoll.edu

Abstract

Embeddings in deep neural networks are essential for processing high-dimensional and categorical data by converting it into compact, low-dimensional vectors. This conversion enables the model to capture complex semantic relationships and improves its generalization across diverse tasks. Although embeddings have demonstrated significant effectiveness in natural language processing and applications involving large language models (LLMs), their utility decreases when handling graph-based data. In this study, we address this challenge by enhancing WalkLM, a cutting-edge language model tailored for generating graph embeddings. We refine the random walk algorithm to better capture both local and global contexts within graphs. Additionally, we implement a k-means algorithm for community detection in these graphs. Our experiments across various benchmark datasets confirm the efficacy of our approach.

CCS Concepts

• Computing methodologies → Neural networks.

Keywords

Graph Clustering, Graph Embedding, Community Detection, Network, Machine Learning, LM(Language Model)

ACM Reference Format:

Hyun Lee, Maminur Islam, Chris Yi, and Chandranil (Nil) Chakrabortii. 2024. Enhancing Graph Representation Learning with WalkLM for Effective Community Detection. In *11th International Conference on Networking, Systems, and Security (NSysS '24)*, December 19–21, 2024, Khulna, Bangladesh. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3704522.3704537>

*Both authors contributed equally to this research.

[†]Corresponding author



This work is licensed under a Creative Commons Attribution International 4.0 License.

NSysS '24, December 19–21, 2024, Khulna, Bangladesh
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1158-9/24/12
<https://doi.org/10.1145/3704522.3704537>

1 Introduction

Large Language Models (LLMs) have revolutionized a wide range of applications in artificial intelligence, especially those related to natural language understanding, generation, translation, summarization, and even code writing. These models, powered by deep neural architectures like transformers[14], have demonstrated exceptional performance in processing and understanding vast amounts of textual data. By leveraging self-attention mechanisms, LLMs can capture intricate relationships between words, phrases, and contexts, excelling in tasks that involve sequential data. This capability has led to significant breakthroughs in conversational agents, content creation, and question-answering systems. The ability of LLMs to generalize across various language-related tasks makes them highly versatile tools in the field of AI.

However, despite their remarkable success in natural language processing (NLP) and related domains, LLMs face significant challenges when dealing with graph-structured data[4][8]. Unlike sequential data, where the relationships between entities follow a linear order, graph data represents complex networks of nodes and edges, where relationships do not adhere to a strict sequence. The non-Euclidean nature of graph topologies, with entities and their interactions spread across a multi-dimensional space, makes it difficult for LLMs to capture the dependencies and relationships inherent in graphs. This limitation renders LLMs less effective for tasks like node classification[19], link prediction, or knowledge graph reasoning—tasks that require a deep understanding of both the local and global structure of graphs.

To address the unique challenges posed by graph-structured data, models like Graph Neural Networks (GNNs)[2] have been developed. GNNs are specifically designed to handle graph data by modeling the dependencies and relationships between nodes and their neighbors in a non-Euclidean space. By aggregating information from a node's neighbors and learning to represent both the node and its connections, GNNs can capture the structural features of a graph, making them suitable for tasks such as node classification[5], link prediction[18], and community detection. Despite their effectiveness, traditional GNNs and Graph Convolutional Networks (GCNs)[5] still have notable limitations. For instance, they often struggle to capture long-range dependencies within the graph[10], leading to poor generalization on large, complex graphs. Additionally, deeper GNN layers are prone to over-smoothing[6],

where node representations become indistinguishable, thereby losing their unique characteristics. Moreover, scalability becomes a concern, as the computational cost of aggregating information from an increasing number of neighbors can grow exponentially with the size of the graph.

In recent years, the combination of graph embeddings[9] and language model-based approaches has emerged as a promising direction for overcoming the limitations of traditional GNNs. Graph embeddings transform nodes, edges, or entire graphs into low-dimensional vectors, while preserving the structural and relational information of the graph. This transformation enables more efficient learning and analysis of graph data, facilitating tasks such as node classification, link prediction, and community detection across various domains, including social networks, biology, and recommendation systems. However, the challenge remains that traditional GNN-based embeddings often fail to capture both local and global contexts effectively[15], particularly in large and intricate graph structures.

Language model-based embeddings, such as those derived from transformers[13], offer a more effective alternative for graph embeddings. Their self-attention mechanisms allow these models to capture long-range dependencies and global structure, addressing the limitations of GNNs. By incorporating the ability to model both structural and semantic relationships, language model-based embeddings can significantly enhance performance on complex graph-related tasks. These embeddings can effectively encode the structural information of the graph while simultaneously capturing the semantic attributes associated with each node, leading to improved results in graph-related tasks such as node classification and link prediction.

In applications such as network analysis, anomaly detection, and cybersecurity, language model-based embeddings can provide significant advantages. For example, they can enhance network analysis tasks by identifying key nodes and detecting anomalous patterns in large-scale networked systems. Additionally, they can support privacy-preserving techniques like federated learning by embedding sensitive data in a way that reduces exposure to raw information, thereby enhancing security and privacy. This flexible and scalable approach makes language model-based embeddings highly valuable for tasks involving privacy, security, and large-scale network applications.

In this context, WalkLM[12] introduces a uniform fine-tuning framework that leverages pre-trained language models (LLMs) to generate attributed graph embeddings. The key innovation of WalkLM lies in its application of random walks on the graph to generate sequences of nodes. These sequences are then fine-tuned with LLMs, enabling the model to capture both the structural (topological relationships between nodes) and semantic (node attributes) information embedded in the graph. By combining random walks with language model fine-tuning, WalkLM achieves significant improvements in various graph-related tasks. For instance, it outperforms traditional GNN-based models in handling both the structure and attributes of graphs, particularly in tasks like node classification and link prediction. Random walks play a crucial role in WalkLM by capturing structural information within the graph. These walks simulate paths through the graph, reflecting neighborhood proximity and the relative importance of nodes. Furthermore, random walks

capture semantic information by encoding node attributes into the sequences, allowing the LLMs to learn both the context and content of the graph's nodes. However, WalkLM's random walk approach primarily captures global context through depth-first search (DFS), which focuses on longer walks to model structural relationships. While this approach is effective in capturing the overall structure of the graph, it lacks the ability to balance local context efficiently.

In contrast, Node2Vec[3] offers a more flexible algorithm by combining DFS and breadth-first search (BFS), enabling the capture of both global and local contexts. While Node2Vec performs well on simpler graphs, it faces limitations when the depth of DFS exceeds 2 in complex graphs, as it struggles to represent intricate topologies effectively.

To overcome these limitations, we propose a novel algorithm that enhances random walks by incorporating the out-degree ratio of neighboring nodes to guide the walk. This approach dynamically adjusts the walk based on the structure of the graph, allowing for a better capture of both global and local contexts in the resulting embeddings. Our algorithm goes beyond the static nature of DFS and BFS in Node2Vec, offering a more adaptable mechanism for embedding. Additionally, we implemented k-means clustering algorithm for community detection, leveraging our enhanced embeddings. Experimental results show that our approach outperforms traditional random walks in WalkLM and biased random walks in Node2Vec, achieving better silhouette scores and producing more meaningful community structures.

2 Related Work

Graph embedding techniques have evolved significantly to address the challenges of representing graph-structured data effectively. Node2Vec and DeepWalk[9] pioneered the use of random walks to generate embeddings, capturing both local and global context through node sequences. Node2Vec employs a flexible strategy that combines depth-first search (DFS) and breadth-first search (BFS) to balance the trade-offs between exploration and exploitation. However, it often struggles with intricate graph topologies, especially when the DFS depth exceeds two levels.

In response to the limitations of shallow embeddings, GNNs have gained prominence. GNNs leverage a message-passing framework to model the relationships between nodes explicitly, allowing for the aggregation of features from neighbors. However, they face challenges like over-smoothing and scalability when applied to large graphs. Variants such as Graph Convolutional Networks (GCNs) have sought to enhance performance but still exhibit issues in capturing long-range dependencies effectively.

Recently, transformer-based models, such as Graph-BERT[17], have emerged, utilizing self-attention mechanisms to improve the capture of complex relationships in graphs. These models offer the potential to encode both structural and semantic information, addressing some limitations of GNNs. Furthermore, efforts to integrate language models with graph data have led to frameworks like WalkLM, which fine-tunes pre-trained language models to generate attributed graph embeddings. This combination of approaches illustrates the rich landscape of research aimed at enhancing graph embeddings, paving the way for advanced applications in areas such

as anomaly detection[1], community detection[11], and network analysis[7].

3 Preliminaries

3.1 Problem Formulation

WalkLM A framework designed to enhance the performance of various downstream tasks such as node classification and link prediction by effectively capturing and representing information from attributed graphs. An attributed graph G is defined as $G = (V, E, \Phi, \Psi)$, where V represents the set of nodes, E denotes the set of edges, and each node $v_i \in V$ and edge $e_i \in E$ is associated with attributes $\Phi(v_i)$ and $\Psi(e_i)$, respectively. The challenge lies in leveraging both the semantic information encapsulated in these attributes and the structural information inherent in the graph's topology. To address this, WalkLM employs an automated textualization program that utilizes attributed random walks to generate meaningful textual sequences $W = W_{i=1}^N$.

3.1.1 Random Walk Textualization in WalkLM. The random walk process in WalkLM textualizes a graph by initiating a walk from a randomly selected node v_0 . The textual representation of this node, denoted as $P(v_0)$, is appended to a sequence W . The walk then extends by randomly selecting an edge e_1 connected to v_0 with a uniform probability, calculated as

$$\frac{1}{\text{number of out-edges of } v_0}.$$

This leads to a terminating node v_1 , whose corresponding texts $P(e_1)$ and $P(v_1)$ are also added to W . The process continues, appending additional nodes and edges, until a termination condition is met, governed by a probability α . The resulting textual sequence

$$W = \{P(v_0), P(e_1), P(v_1), \dots, P(v_{L-1}), P(e_L), P(v_L)\}$$

reflects a random walk of length $2L + 1$ on the graph. This sequence forms a meaningful representation, akin to a sentence, that captures both structural and semantic information from the graph.

By repeating this process N times, WalkLM generates a collection of N attributed random walks

$$W = \{W_i\}_{i=1}^N,$$

creating a graph-aware corpus. This corpus is used to train language models without requiring supervision for downstream tasks, enabling the model to learn from the intrinsic relationships and attributes present in the graph.

Once the graph has been transformed into these sequences, WalkLM proceeds to fine-tune a graph-aware language model (LM) using the Masked Language Modeling (MLM) technique. This approach is critical for enabling the model to learn robust representations from the generated graph-aware sequences. In the MLM process, 15 percent of the tokens in each sequence are randomly masked, and the model is tasked with predicting the original tokens. This predictive mechanism fosters an understanding of contextual relationships, even when certain tokens are missing or altered. Following fine-tuning, WalkLM optimizes the parameters Θ of the LM, allowing it to effectively learn from the graph-aware data. The resulting embedding vectors extracted from the LM encapsulate

both structural and semantic characteristics of the graph. These embeddings are subsequently evaluated in various downstream tasks, demonstrating significant performance improvements by integrating graph-based textual sequences into the model's training process, thereby enhancing the efficacy of node classification and link prediction tasks.

$$L_{\text{MLM}} = -\frac{1}{|X|} \sum_{X \in X} \sum_{t_i \in M} \log p(t_i | T_{\setminus i}),$$

where X is the set of training examples, M is the set of masked tokens, and $T_{\setminus i} = \{t_1, \dots, t_{i-1}, t_{i+1}, \dots, t_L\}$ is the context around t_i . This loss function promotes the model's ability to discern relationships between tokens, even when certain tokens are absent or modified, resulting in a more robust and generalizable language model. By employing this technique on graph-based sequences, WalkLM effectively captures both the syntactic and structural dependencies present in the sequences derived from random walks.

Subsequently, WalkLM fine-tunes a graph-aware language model (LM) utilizing the random walk-based textual sequences W . During this fine-tuning process, the model optimizes its parameters Θ , enabling it to learn efficiently from the graph-aware data. Upon completion of the fine-tuning, the embedding vectors obtained represent learned interpretations of the graph structure. These embeddings are then utilized in downstream tasks τ_1 and τ_2 , where the performance enhancements resulting from the integration of graph-based textual sequences into the model's training regimen can be assessed.

3.1.2 Biased Random Walk in Node2Vec. In Node2Vec[3], the biased random walk algorithm is designed to capture both local and global structural features of a graph by adjusting the likelihood of visiting nodes based on two parameters: p and q . These parameters help determine the nature of the walk and influence how the random walk behaves when transitioning between nodes. Specifically, a higher value of p reduces the likelihood of revisiting previously visited nodes, thereby encouraging exploration of new nodes. Conversely, a lower value of q increases the tendency to visit new nodes. By adjusting the return parameter p and the in-out parameter q , we can tailor the sampling process to capture the desired characteristics of the network. This flexibility allows for a more nuanced exploration of the graph's structure, making it more suitable for tasks that require an understanding of both local and global features.

For example, consider a random walk that just traversed edge (t, v) and now resides at node v (Figure 1). The walk now needs to decide on the next step so it evaluates the transition probabilities π_{vx} on edges (v, x) leading from v . The edge weight w_{vx} is defined as the weight of the edge connecting node v to node x . For unweighted graphs, $w_{vx} = 1$. The simplest way to bias our random walks would be to sample the next node based on the static edge weights w_{vx} , i.e., $\pi_{vx} = w_{vx}$. We set the unnormalized transition probability to $\pi_{vx} = \alpha_{pq}(t, x) \cdot w_{vx}$, where

$$\alpha_{pq}(t, x) = \begin{cases} \frac{1}{p} & \text{if } d_{tx} = 0 \\ 1 & \text{if } d_{tx} = 1 \\ \frac{1}{q} & \text{if } d_{tx} = 2 \end{cases}$$

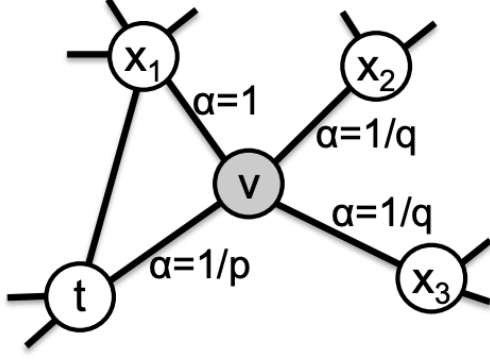


Figure 1: Illustration of the random walk procedure in node2vec. The walk just transitioned from t to v and is now evaluating its next step out of node v . Edge labels indicate search biases α .

and d_{tx} denotes the shortest path distance between nodes t and x . Note that d_{tx} must be one of $\{0, 1, 2\}$, and hence, the two parameters are necessary and sufficient to guide the walk. Intuitively, parameters p and q control how fast the walk explores and leaves the neighborhood of starting node u . In particular, the parameters allow our search procedure to (approximately) interpolate between BFS and DFS and thereby reflect an affinity for different notions of node equivalences.

4 Hybrid Textualization Methodology

Our hybrid approach harnesses the strengths of both WalkLM and Node2Vec, while simultaneously addressing the limitations of each in capturing graph locality and community structures. WalkLM’s random walk-based textualization excels at converting graph data into meaningful text, effectively capturing both topological structures and attribute information. However, it often falls short in understanding graph locality. This makes it challenging to distinguish nodes that are geographically or structurally close within a graph but belong to different communities. As a result, WalkLM struggles to detect nuanced community structures, particularly in complex, real-world graphs. Despite capturing attribute-based information, its lack of graph-specific insights limits its effectiveness in embedding graphs for community detection.

Conversely, Node2Vec balances local and global context by blending BFS (Breadth-First Search) and DFS (Depth-First Search), utilizing its tunable parameters p and q to explore neighborhoods effectively. However, the depth of exploration is limited to a maximum of 2, which is often insufficient for capturing deeper, more intricate relationships within complex graph data. This restricted exploration hinders the model’s ability to fully grasp the broader context necessary for effective community detection.

To tackle these challenges, we propose an outgoing degree-based BFS path exploration technique. The outgoing degree of a node, denoted as $\text{deg}_{\text{out}}(v)$, is defined as the number of edges directed away from the node v . The intuition behind this method is that when the ratio of the outgoing degrees between two nodes is high,

these nodes likely belong to different communities and have significant roles within their respective clusters. By integrating this degree-based exploration into the embedding process, our approach captures more subtle structural features of the graph, leading to improved community detection.

Algorithm 1 BFS with Maximum Hops and Outgoing Degree Threshold

Require: Graph G , Maximum Hops max_hops , Threshold β
Ensure: List of all BFS paths from every node, up to max_hops

```

1: function BFS_WITH_CONSTRAINTS( $G, \text{max\_hops}, \beta$ )
2:   Initialize  $\text{bfs\_paths} \leftarrow []$ 
3:   for each node  $v$  in  $G$  do
4:     Initialize queue  $Q$ 
5:     Enqueue  $(v, [v], 0)$  into  $Q$ 
6:     while  $Q$  is not empty do
7:        $(\text{current\_node}, \text{current\_path}, \text{level}) \leftarrow Q.\text{dequeue}()$ 
8:       if  $\text{level} \geq \text{max\_hops}$  then
9:         continue
10:      end if
11:      for each neighbor  $u$  of  $\text{current\_node}$  in  $G$  do
12:         $\text{ratio} \leftarrow \frac{|\text{Adjacency List of current\_node}|}{|\text{Adjacency List of } u|}$ 
13:        if  $\text{ratio} > \beta$  then
14:           $\text{new\_path} \leftarrow \text{current\_path} + [u]$ 
15:          Enqueue  $(u, \text{new\_path}, \text{level} + 1)$  into  $Q$ 
16:        end if
17:      end for
18:    end while
19:    Add  $\text{current\_path}$  to  $\text{bfs\_paths}$ 
20:  end for
21:  return  $\text{bfs\_paths}$ 
22: end function

```

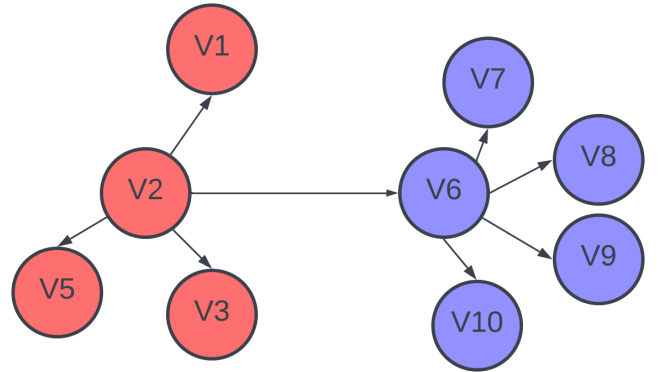


Figure 2: Illustration of how out-degree information can affect the generation of graph embedding.

Figure 2 illustrates two distinct communities within the graph. One community consists of nodes V_1, V_2, V_3 , and V_5 , while the other comprises nodes V_6, V_7, V_8, V_9 , and V_{10} . When our BFS algorithm

starts from node V_2 , it prioritizes the nodes it explores based on the out-degree ratios.

In contrast, the WalkLM method would treat V_1 and V_6 with equal importance. However, this approach overlooks the fact that V_1 has no outgoing edges, whereas V_6 has multiple outgoing edges. Therefore, our outgoing degree ratio-based algorithm assigns a higher preference to selecting the path from V_2 to V_6 in the next exploration level, as V_6 contributes more significantly to the structural relationships of the graph. This nuanced prioritization allows our algorithm to more effectively capture the underlying community structure, unlike random walks, which fail to differentiate between V_1 and V_6 and thus treat them as equally important. Consequently, our approach enhances the exploration of the graph by focusing on nodes that have a greater influence on the structural relationships, leading to improved clustering and community detection.

4.1 Silhouette score for community detection

Our approach employs the Hybrid Walk algorithm to generate a text corpus, which we then use to learn the graph's representation or embedding. Subsequently, we apply K-means clustering to identify clusters or communities within the graph. To evaluate the effectiveness of this community detection method, we utilize the silhouette score, a widely recognized metric for assessing cluster quality. This measure proves particularly valuable when analyzing real-world graph datasets that lack ground truth labels.

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad (1)$$

Where $a(i)$ is the average distance between node i and all other nodes in its cluster, and $b(i)$ is the average distance between node i and all nodes in the nearest cluster to which i does not belong.

5 Experiments

5.1 Experimental Setup

Datasets

We implemented our algorithm based on two different real world datasets, PubMed and Cisco22. PubMed contains a graph of genes, diseases, chemicals, and species. The nodes and edges are extracted according to [16]. A relatively small fraction of diseases are grouped into eight categories.

Table 1: Statistics of PubMed Dataset

Dataset	#attribute type	#node type	#node	#link type	#link	#label	#label node
PubMed	8	4	63,109	10	244,986	8	454

The Cisco22 dataset represents a network graph where edges denote connections between client nodes (consumers) and server nodes (providers), with attributes like server ports and communication protocols (6 for TCP, 17 for UDP).

5.2 Experimental Results on PubMed Dataset

In our experiment, for the graph embedding, we conducted three different experiments for the graph textualization process. First, we performed these experiments using the *Random_Walk* (RW) algorithm in WalkLM, an out-degree ratio based BFS algorithm referred

Table 2: Statistics of Cisco-22 Dataset

Graph	#Nodes	Med. Deg.	Max Deg.	#Ports	#M-Ports	#Dir. Edges
g4	161,777	1	287	54	1	175,610
g2	109,634	2	89,168	69,310	2	1,597,194
g5	45,876	1	2,837	418	1	54,658
g15	45,706	2	1,246	974	1	89,295
g6	24,032	2	4,211	680	1	102,153
g10	12,614	2	5,348	4,259	1	27,840
g21	52	50	2,371	1,398	8	2,507
g21'	52	21	2,369	1,396	6	952

to as *BFS_Walk*, and a Hybrid method which is referred to as *Hybrid_Walk* that utilizes both the Random Walk and the out-degree ratio based BFS algorithm. These approaches were empirically evaluated to assess their relative performance in various downstream tasks. The effectiveness of the generated graph embeddings was assessed through a link-prediction task. To ensure the reliability of our findings, each experiment was repeated five times. The average accuracy percentages achieved for the Random Walk, BFS-Walk, and Hybrid Walk methods were 87%, 87%, and 89%, respectively. These results are visually represented in Figure 3.

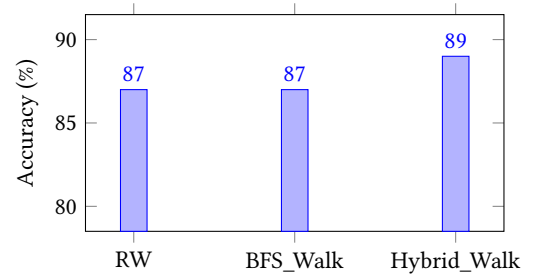


Figure 3: Link prediction Accuracy of graph embeddings for different methods on PubMed dataset.

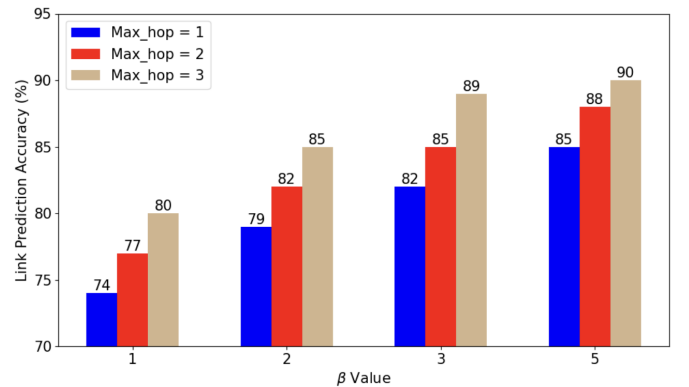


Figure 4: Link prediction accuracy for different β values and Max_hop settings.

As shown in Figure 3, link prediction accuracy varies with β values and Max_hop settings. Max_hop determines the maximum

number of hops considered in graph traversal. Among the configurations tested, the Max_hop=3 setting achieved the highest accuracy, peaking at 90% when $\beta = 5$. This combination provides the best results within the experimental setup, balancing out-degree ratio and computational cost.

For lower β values, the text corpus is insufficient to provide meaningful contextual information. Conversely, increasing Max_hop beyond 3 results in significantly higher computational resource requirements, exceeding the capabilities of the experimental setup, which utilized an NVIDIA 4080 GPU 16GB. Additionally, smaller Max_hop values fail to capture sufficient structural information, leading to suboptimal performance.

Higher Max_hop settings also introduce noise into the model due to the inclusion of distant connections that may not be directly relevant to the target nodes. These distant connections can dilute the influence of closer, more meaningful relationships, leading to a less focused and effective representation of the graph's structure. As such, the combination of Max_hop=3 and $\beta = 5$ is not universally optimal but represents the best configuration achievable within the scope of our experimental parameters.

5.3 Experimental Results on Cisco-22 Dataset

We applied a textualization process algorithm on the Cisco dataset to generate graph embeddings. The resulting embeddings were then projected onto a two-dimensional space using t-SNE (t-Distributed Stochastic Neighbor Embedding), which facilitates visualization and analysis of high-dimensional data. The outcomes of this process are illustrated in Figure 5, demonstrating clear clusters that indicate well-defined community structures within the dataset.

We evaluated performance with the silhouette score, comparing it against WalkLM with RW, our BFS_Walk algorithm, and Hybrid_Walk.

The experimental results illustrated that the BFS_Walk algorithm outperformed the RW method in terms of graph clustering. This finding underscores the significance of considering local information for effective graph clustering. Furthermore, our Hybrid_Walk algorithm, which adeptly balances global and local information, demonstrated approximately 4% higher performance compared to the standalone RW and BFS_Walk methods. This indicates that an integrated approach leveraging both global and local information is essential for enhancing graph clustering tasks.

These results, presented as silhouette scores, are summarized in Table 3, showcasing the comparative performance of each method.

Table 3: Silhouette scores for graph clustering on the Cisco-22 dataset using different algorithms. The higher silhouette score for Hybrid_Walk underscores its superior performance.

Algorithm	Silhouette Score
RW	0.35
BFS_Walk	0.36
Hybrid_Walk	0.40

6 Conclusion

Graph representation learning has advanced significantly, primarily through the development of Graph Neural Networks (GNNs) and

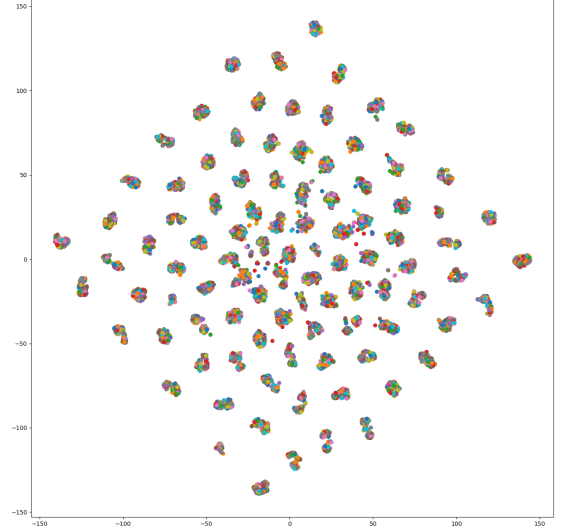


Figure 5: Graph embeddings generated from the Cisco dataset after applying the textualization process algorithm, projected using t-SNE. The figure illustrates distinct clusters, highlighting the effectiveness of the embeddings for community detection.

their application in various domains such as node classification and community detection. Among these, graph clustering remains a crucial area of research, with its ability to reveal hidden structures and patterns in diverse real-world applications, including fraud detection and social network analysis. Despite their successes, existing GNN-based methods often struggle with challenges such as oversmoothing, limited capacity for directed graphs, and an inadequate focus on node features, which can compromise clustering effectiveness.

To tackle these issues, our proposed hybrid walk approach leverages both random walks and degree-based exploration, providing a robust method for attributed graph clustering. By integrating fine-tuning techniques and enhancing representation learning, the hybrid walk ensures that learned embeddings retain essential structural information, improving clustering performance. Our experiments across benchmark datasets validate the efficacy of our method, showcasing its potential to address current limitations in graph clustering techniques.

As part of our future work, we would like to develop novel effective algorithms for community detection, enhanced LLM-based graph representation learning, and robust path exploration for the textualization of graphs, further advancing the capabilities of graph-based methods in various applications.

7 Future Work

Building on the promising results of our hybrid walk approach for attributed graph clustering, there are several avenues for future research to enhance graph-based methods further. First, expanding the methodology to address community detection, including overlapping and hierarchical structures, can provide deeper insights into modular graph properties. Leveraging advancements in large language models (LLMs) for graph representation learning offers an opportunity to improve contextual understanding of graph structures and node attributes through graph textualization. Additionally, robust path exploration methods can enhance the generation of meaningful textual representations of graphs for downstream applications. Future work will also focus on scalability to handle large-scale graphs using distributed computing techniques and approximate algorithms. Extending the approach to directed and dynamic graphs, capturing temporal changes and asymmetric relationships, is crucial for applications such as social network analysis and traffic prediction. Integrating graph contrastive learning techniques could further refine embeddings by balancing local and global graph information. Lastly, applying the hybrid walk approach to diverse domains such as biology, e-commerce, and cybersecurity will demonstrate its adaptability and effectiveness across real-world scenarios. These directions aim to address current limitations and push the boundaries of graph representation learning and clustering methods.

References

- [1] Leman Akoglu, Hanghang Tong, and Danai Koutra. 2014. Graph-based Anomaly Detection and Description: A Survey. arXiv:1404.4679 [cs.SI] <https://arxiv.org/abs/1404.4679>
- [2] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. 2017. Neural Message Passing for Quantum Chemistry. arXiv:1704.01212 [cs.LG] <https://arxiv.org/abs/1704.01212>
- [3] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable Feature Learning for Networks. arXiv:1607.00653 [cs.SI] <https://arxiv.org/abs/1607.00653>
- [4] Jiayan Guo, Lun Du, Hengyu Liu, Mengyu Zhou, Xinyi He, and Shi Han. 2023. GPT4Graph: Can Large Language Models Understand Graph Structured Data? An Empirical Evaluation and Benchmarking. arXiv:2305.15066 [cs.AI] <https://arxiv.org/abs/2305.15066>
- [5] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. arXiv:1609.02907 [cs.LG] <https://arxiv.org/abs/1609.02907>
- [6] Qimai Li, Zhichao Han, and Xiao-Ming Wu. 2018. Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning. arXiv:1801.07606 [cs.LG] <https://arxiv.org/abs/1801.07606>
- [7] Yuhao Li, Zhixun Li, Peisong Wang, Jia Li, Xiangguo Sun, Hong Cheng, and Jeffrey Xu Yu. 2024. A Survey of Graph Meets Large Language Model: Progress and Future Directions. arXiv:2311.12399 [cs.LG] <https://arxiv.org/abs/2311.12399>
- [8] Yixin Liu, Ming Jin, Shirui Pan, Chuan Zhou, Yu Zheng, Feng Xia, and Philip Yu. 2022. Graph Self-Supervised Learning: A Survey. *IEEE Transactions on Knowledge and Data Engineering* (2022), 1–1. <https://doi.org/10.1109/tkde.2022.3172903>
- [9] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. DeepWalk: online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining (KDD '14, Vol. 2)*. ACM, 701–710. <https://doi.org/10.1145/2623330.2623732>
- [10] Xing Su, Shan Xue, Fanzhen Liu, Jia Wu, Jian Yang, Chuan Zhou, Wenbin Hu, Cecile Paris, Surya Nepal, Di Jin, Quan Z. Sheng, and Philip S. Yu. 2024. A Comprehensive Survey on Community Detection With Deep Learning. *IEEE Transactions on Neural Networks and Learning Systems* 35, 4 (April 2024), 4682–4702. <https://doi.org/10.1109/tnnls.2021.3137396>
- [11] Xing Su, Shan Xue, Fanzhen Liu, Jia Wu, Jian Yang, Chuan Zhou, Wenbin Hu, Cecile Paris, Surya Nepal, Di Jin, Quan Z. Sheng, and Philip S. Yu. 2024. A Comprehensive Survey on Community Detection With Deep Learning. *IEEE Transactions on Neural Networks and Learning Systems* 35, 4 (2024), 4682–4702. <https://doi.org/10.1109/TNNLS.2021.3137396>
- [12] Yanchao Tan, Zihao Zhou, Hang Lv, Weiming Liu, and Carl Yang. 2023. WalkLM: A Uniform Language Model Fine-tuning Framework for Attributed Graph Embedding. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 13308–13325. https://proceedings.neurips.cc/paper_files/paper/2023/file/2ac879d1865475a7abc8dfc7a9c15c27-Paper-Conference.pdf
- [13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need. arXiv:1706.03762 [cs.CL] <https://arxiv.org/abs/1706.03762>
- [14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [15] Felix Wu, Tianyi Zhang, Amauri Holanda de Souza Jr. au2, Christopher Fifty, Tao Yu, and Kilian Q. Weinberger. 2019. Simplifying Graph Convolutional Networks. arXiv:1902.07153 [cs.LG] <https://arxiv.org/abs/1902.07153>
- [16] Carl Yang, Yuxin Xiao, Yu Zhang, Yizhou Sun, and Jiawei Han. 2020. Heterogeneous Network Representation Learning: A Unified Framework with Survey and Benchmark. *IEEE Transactions on Knowledge and Data Engineering* 34, 10 (2020), 4854–4873. <https://doi.org/10.1109/TKDE.2020.2981334>
- [17] Jiawei Zhang, Haopeng Zhang, Congying Xia, and Li Sun. 2020. GraphBert: Only Attention is Needed for Learning Graph Representations. arXiv:2001.05140 [cs.LG] <https://arxiv.org/abs/2001.05140>
- [18] Muhao Zhang and Yixin Chen. 2018. Link Prediction Based on Graph Neural Networks. arXiv:1802.09691 [cs.LG] <https://arxiv.org/abs/1802.09691>
- [19] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. 2020. Graph neural networks: A review of methods and applications. *AI Open* 1 (2020), 57–81. <https://doi.org/10.1016/j.aiopen.2021.01.001>